

AI Capstone HW5 Report

Group 08

NOT DONE YET!!!

1. Repository Contents

The repository follows the required submission structure specified in Section 16.3.

In this project, we primarily utilized **Protégé** for ontology construction, reasoning, and SPARQL query execution. To complement these native features, we implemented a **Python-based** workflow to perform additional reasoning and queries, generating well-structured and highly readable tables. Furthermore, we integrated **Widoco** to enhance the overall documentation and readability of our ontology.

1-1. Ontologys (ontology/)

This folder contains the main authored ontology and all required ontology-related artifacts.

- `group-ontology.ttl`

The main ontology developed by Group 08.

It's authored in Protégé and exported as a .ttl file.

- `inferred-results-protege.ttl`

The output graph generated after OWL reasoning.

Contains two inferred class memberships, `cap:GraspableObject` and `g08:ColorSortableObject`.

It's generated from Protégé and exported as a .ttl file.

- `inferred-results-python.ttl`

The output graph generated from `run_reasoning.py`.

- `ontology/imports/course-affordance.ttl`

The required shared vocabulary provided by the course.

1-2. Source Code (src/)

- `run_reasoning.py`

In addition to using Protégé's built-in reasoner and SPARQL plugin, we implemented a Python script to automate the reasoning and SPARQL query process. This allowed us to generate a more clearly classified and structured table of the results.

1-3. SPARQL Queries (queries/)

This folder contains SPARQL queries used to evaluate and validate the ontology under different reasoning and analysis purposes.

- `queries/graspable_objects.rq`

This is the required baseline query for the assignment.

It retrieves all inferred `cap:GraspableObject` individuals after reasoning.

This query is used in both Protégé and the Python reasoning pipeline for verification of inference results.

- `queries/color_sortable_objects.rq`

This query is designed for the group's extended task involving color-based sorting.

It retrieves individuals classified under the extended concept of color-sortable objects.

This query is primarily executed in Protégé for inspection and validation of the advanced ontology design.

- `queries/task_objects.rq`

This query is used within the Python reasoning workflow.

It serves as a preprocessing step to filter relevant task-related individuals before generating final output tables.

This query is not primarily used in Protégé, but supports automated pipeline execution in `run_reasoning.py`.

1-4. Query and Widoco Results (results/)

This folder stores outputs generated from executing SPARQL queries under reasoning-enabled conditions.

- `protege_graspable_objects_output.txt`
- `protege_graspable_objects_output_screenshot.png`

- `python_graspable_objects_output.txt`

Output of baseline requirement `graspable_objects.rq`.

Used to verify inferred instances of `cap:GraspableObject`.

- `protege_color_sortable_objects_output.txt`
- `protege_color_sortable_objects_output_screenshot.png`

Output of `color_sortable_objects.rq`.

Used to validate the extended color-sorting reasoning results in Protégé.

- `python_task_objects_output.txt`

Output of `run_seasoning.py`.

Used internally by the Python pipeline for constructing final structured result tables.

- `widoco_screenshot_crossref.png`
- `widoco_screenshot_index.png`

The screenshot of Widoco.

1-5. Widoco (`doc/widoco/doc/`)

This folder contains the ontology documentation generated using WIDOCO.

Automatically generated HTML documentation of the ontology.

Provides human-readable browsing of classes, properties, and individuals.

Includes ontology metadata, diagrams, and term descriptions.

1-6. Python Environment Requirements

- `requirements.txt`

Specifies Python dependencies required to execute `run_reasoning.py`.

2. Ontology Design Rationale

The ontology is primarily designed based on the structure and semantics defined in the provided course ontology `course-affordance.ttl`.

The ontology is designed to represent robot manipulation scenarios in which a robot interacts with physical objects under different task contexts.

we adopt a generic design based on four main concepts:

- Physical objects
- Affordances
- Tasks
- Task roles

This design allows the same object to participate in different tasks while taking different semantic roles.

The affordance-centered design also enables reasoning over object capabilities instead of relying solely on object categories.

3. Namespace Policy

The ontology follows a two namespaces strategy as spec requirement.

Course Ontology Namespace

cap:

Contains core concepts provided by the course ontology and shared across the project.

g08:

Contains concepts introduced by Group 08.

This separation clearly distinguishes reused concepts from group-specific extensions.

Note on Namespace: In our implementation, GraspableObject was intentionally assigned to the local group namespace (g08:GraspableObject) rather than the cap: prefix. This design choice was maintained throughout the development pipeline to preserve the integrity of our generated graphs and query results. By localizing the class under g08:, we ensure that our team's specific extensions and reasoning workflows remain modular and decoupled from the shared core vocabulary, preventing any potential namespace pollution.

4. Ontology Terms: Reused and Newly Introduced

To ensure semantic consistency and enable group-specific reasoning extensions, we strictly separate

the vocabularies in our repository into **Reused Terms** (adopted from the core course ontology and standard semantic specifications) and **Newly Introduced Terms** (classes and instances customized by Group 08).

4.1 Reused Terms

Reused terms include the foundational building blocks of the project structure, consisting of the explicit course ontology requirements and global W3C standards.

1. Core Course Ontology Classes (cap:)

We reused the pre-defined environment and task-specific concepts to construct our local world model:

Abstract & Structural Classes: cap:PhysicalObject, cap:Affordance, cap:TaskRole, cap:Basket.

Task-Specific Object Classes: cap:Cup, cap:Knife, cap:Fork, cap:Plate, cap:ToyBlock.

Affordance Profiles: cap:GraspingAffordance, cap:ContainmentAffordance, cap:StackabilityAffordance, cap:SupportAffordance.

Task Roles: cap:TargetObject, cap:ReferenceObject, cap:ContainerTarget, cap:CollectableObject.

2. Core Course Ontology Properties (cap:)

To link our physical entities with their semantic roles and perceptual observations, we reused the following properties directly:

Object Properties: cap:hasTaskRole, cap:hasAffordance, cap:hasTargetObject, cap:hasReferenceObject, cap:canBeManipulatedBy.

Datatype Properties: cap:hasColor, cap:hasObjectLabel, cap:hasPoseFrame, cap:hasApproxWidth.

3. Standard Semantic Web Vocabularies

Universal W3C terms were reused to maintain proper structural metadata and data typing:

OWL/RDF Primitives: rdf:type, rdfs:subClassOf, rdfs:label, rdfs:comment, owl:Class, owl:ObjectProperty, owl:DatatypeProperty, owl:Restriction, owl:equivalentClass, owl:intersectionOf.

Metadata & Types: dcterms:creator, dcterms:created, dcterms:description, dcterms:title, dcterms:license, vann:preferredNamespacePrefix, vann:preferredNamespaceUri, foaf:Person, foaf:name, foaf:mbox, and xsd:string, xsd:decimal, xsd:date.

4.2 Newly Introduced Terms

Newly introduced terms represent the customized reasoning extensions and concrete physical instances generated locally under our group's namespace (g08:).

1. Group-Extended Classes (Reasoning & Task Expansion)

To support advanced automated classification via our Python pipeline and Protégé reasoners, we introduced:

g08:GraspableObject: An inferable class defined as an intersection of cap:PhysicalObject and an existential restriction on cap:hasAffordance some cap:GraspingAffordance. (Note: Managed under our local prefix for system verification and query formatting).

g08:ColorSortingAffordance: A subclass of cap:Affordance capturing an object's action capability to be sorted into a matching container based on visual color traits.

g08:ColorSortableObject: An inferable class defined as an intersection of cap:PhysicalObject and an existential restriction on cap:hasAffordance some g08:ColorSortingAffordance.

g08:ColorSortingBasket: A subclass of cap:Basket constrained to possess a g08:ColorSortingAffordance for color-based container routing.

2. Local Grounded Individuals

We populated the ontology with 12 specific grounded individuals to evaluate our semantic mapping and reasoning workflows under different simulation scenes:

Cutlery Task: g08:fork01 (Target), g08:knife01 (Target), g08:plate01 (Reference).

Cup Stacking Task: g08:blueCup01 (Target, Blue), g08:pinkCup01 (Reference, Pink).

Baseline Block Task: g08:baselineBasket01 (Container Target).

Color Sorting & Collection Task:

Blocks: g08:redBlock01 (Red), g08:blueBlock01 (Blue), g08:greenBlock01 (Green).

Baskets: g08:redBasket01 (Red), g08:blueBasket01 (Blue), g08:greenBasket01 (Green).

5. Key Axioms and Restrictions

To enable automated classification and semantic grounding, our ontology defines several key theological axioms and logical restrictions using OWL primitives. These rules allow the reasoner to infer implicit relationships from perceived object properties.

4.1 Equivalent Class Axioms and Existential Restrictions

We implemented Defined Classes (anonymous class expressions) using owl:equivalentClass combined with owl:intersectionOf and owl:someValuesFrom restrictions. This allows the reasoner to dynamically classify objects based on their manifested affordances:

Inference Rule for Graspable Objects (g08:GraspableObject)

Logical Definition: A g08:GraspableObject is defined as any cap:PhysicalObject that has at least one cap:GraspingAffordance linked via the cap:hasAffordance property.

OWL Expression: cap:PhysicalObject AND (cap:hasAffordance some cap:GraspingAffordance)

Inference Rule for Color-Sortable Objects (g08:ColorSortableObject)

Logical Definition: A g08:ColorSortableObject is defined as any cap:PhysicalObject that exhibits a g08:ColorSortingAffordance via the cap:hasAffordance property.

OWL Expression: cap:PhysicalObject AND (cap:hasAffordance some g08:ColorSortingAffordance)

4.2 Subclass Axioms and Property Domain/Range Constraints

We maintained structural integrity and inheritance boundaries through explicit subclass axioms and property constraints inherited from the course specification:

Subclass Axioms:

g08:ColorSortingBasket is strictly asserted as a subclass of cap:Basket.

g08:ColorSortingAffordance is strictly asserted as a subclass of cap:Affordance.

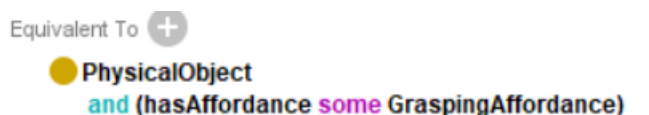
Value Restrictions:

g08:ColorSortingBasket is restricted to only receive individuals that possess a g08:ColorSortingAffordance (owl:someValuesFrom g08:ColorSortingAffordance), ensuring that color-sorting operations are constrained to appropriate target containers.

7. Reasoning Pattern

The ontology uses OWL equivalent class definitions to enable automatic classification.

GraspableObject



Any physical object possessing a grasping affordance can be automatically inferred as a graspable

object.

ColorSortableObject

Equivalent To 

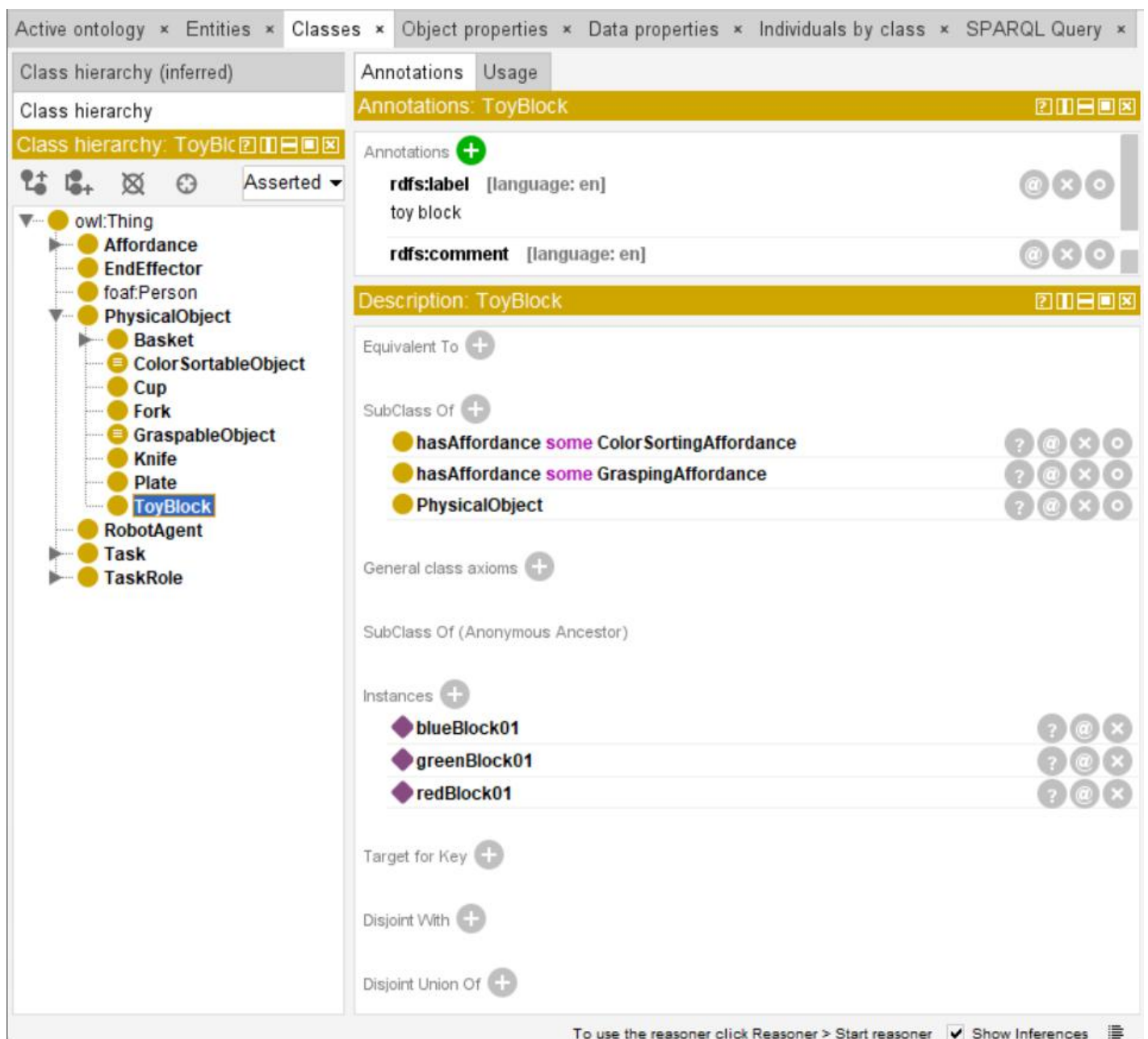
 **PhysicalObject**
and (hasAffordance  **ColorSortingAffordance**)

Any physical object possessing a color-sorting affordance can be automatically inferred as a color-sortable object.

8. Reasoning Results

After running an OWL reasoner, additional class memberships are inferred automatically.

Before Reasoning (take toyBlock as an example):



The screenshot shows the Protégé OWL editor interface. The top navigation bar includes tabs for 'Active ontology', 'Entities', 'Classes', 'Object properties', 'Data properties', 'Individuals by class', and 'SPARQL Query'. The 'Classes' tab is active, displaying the 'Class hierarchy (inferred)' on the left and the 'Annotations' and 'Usage' panels on the right.

Class hierarchy (inferred):

- owl:Thing
 - Affordance
 - EndEffector
 - foaf:Person
 - PhysicalObject
 - Basket
 - ColorSortableObject**
 - Cup
 - Fork
 - GraspableObject
 - Knife
 - Plate
 - ToyBlock**
 - RobotAgent
 - Task
 - TaskRole

After Reasoning:

Active ontology × Entities × Classes × Object properties × Data properties × Individuals by class × SPARQL Query ×

Class hierarchy (inferred)

Class hierarchy

Class hierarchy: ToyBlock ? [] [] [] [] []

Annotations

Annotations: ToyBlock ? [] [] [] [] []

Annotations +

rdfs:label [language: en]
toy block

Description: ToyBlock ? [] [] [] [] []

Equivalent To +

SubClass Of +

- hasAffordance some ColorSortingAffordance
- hasAffordance some GraspingAffordance
- PhysicalObject
- ColorSortableObject
- GraspableObject

General class axioms +

SubClass Of (Anonymous Ancestor)

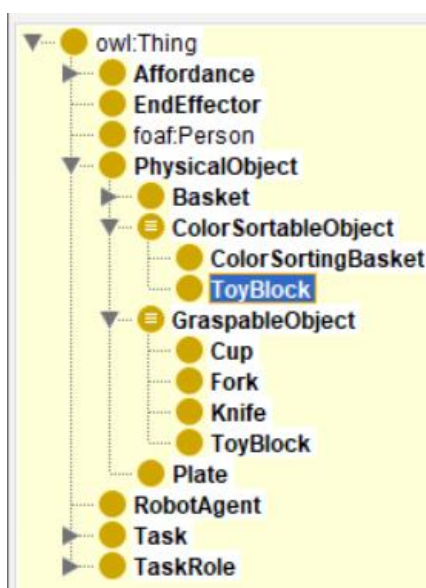
- PhysicalObject and (hasAffordance some GraspingAffordance)
- PhysicalObject and (hasAffordance some ColorSortingAffordance)

Instances +

- blueBlock01
- greenBlock01
- redBlock01

Target for Key +

Reasoner active ☒ Show Inferences



9. Query Results

Example SPARQL queries were executed after reasoning.

Query 1: Retrieve All Graspable Objects

```
```sparql
SELECT ?obj
WHERE {
 ?obj rdf:type g08:GraspableObject .
}
```
```

Result:

- * redBlock01
- * greenBlock01
- * blueBlock01
- * pinkCup01
- * blueCup01
- * fork01
- * knife01

Query 2: Retrieve All Color-Sortable Objects

```
```sparql
SELECT ?obj
```

```
WHERE {

 ?obj rdf:type g08:ColorSortableObject .

}
...
```

Result:

```
* redBlock01

* greenBlock01

* blueBlock01
```

The query results confirm that the intended reasoning patterns operate correctly.

---

## ## 10. Design Choices

Several design choices were made during ontology development.

### ### Affordance-Centered Modeling

Instead of encoding capabilities directly into object classes, capabilities are represented as affordances.

This allows different object categories to share common behaviors.

For example:

\* ToyBlock

\* Cup

\* Fork

\* Knife

all share the same `GraspingAffordance`.

### ### Separation Between Object Type and Task Role

Object type and task role are modeled independently.

For example, a basket is a physical object category, while `ContainerTarget` represents the role played in a specific task.

This improves ontology flexibility and reuse.

### ### Reasoning-Based Classification

Equivalent class definitions are used to support automatic classification.

This reduces manual maintenance and demonstrates the usefulness of OWL reasoning.

---

## ## 11. Limitations

The ontology focuses primarily on symbolic task representation and reasoning.

Several limitations remain.

### ### Geometric Information

The ontology does not represent:

- \* collision relationships
- \* object geometry
- \* object shape models

### ### Motion Planning

The ontology does not model:

- \* robot trajectories
- \* grasp poses
- \* path planning constraints

### ### Temporal Information

The ontology does not represent:

- \* task sequences
- \* execution order
- \* temporal constraints

### ### Color Matching Logic

Although color information is represented using the `hasColor` property, OWL reasoning does not currently enforce that blocks must be placed into baskets of matching colors.

Such constraints would require additional rules or external reasoning mechanisms.

---

## ## 12. Conclusion

Group 08 extends the baseline ontology with affordance-based reasoning capabilities and a color-sorting task extension.

The ontology demonstrates how OWL restrictions and equivalent class definitions can be used to infer higher-level semantic categories automatically. The resulting model remains reusable across multiple manipulation scenarios while supporting meaningful reasoning and querying over task-relevant object properties.